Lab Prompt & Search

Modulo M2 · Lezione 24



Da dove si riparte

Sabato scorso (L23) il RAG completo del corso era pronto e funzionante.

quello che c'è già

`working/rag.py` : `cerca_chunk` · `costruisci_prompt` · `rispondi` · la collection `lumen_chunks`

Oggi NON si scrive

un RAG nuovo

Oggi si MISURA

quello che c'è, una manopola alla volta

> Tutte le manopole viste finora — **prompt**, **k e soglia**, **chunking** — girate oggi **una alla volta**, con una scheda davanti.



Agenda

Nessun live coding oggi: il docente parla solo nel brief e nell'ultimo blocco.
⌨️ = scrivete voi (docente in affiancamento) · 📊 = slide

	Blocco	Dove si lavora	Tempo
📊	Brief: il metodo, la query set, la scheda	slide	25′
⌨️	**Lab 1: A/B di prompt**	`working/rag.py`	90′
☕	Pausa	—	15′
⌨️	**Lab 2: tuning del retrieval**	`working/rag.py`	80′
📊⌨️	**Ultimo blocco** (a scelta del docente)	slide / `working/rag.py`	30′

> Setup passo-passo in `scripts/CONSEGNA.md` : stessa chiave OpenAI di L13-L23, propria collection/ `rag.py` di L22-L23 (chi non li ha: riserva, un comando).



Il metodo: una manopola alla volta

La regola della giornata

Si cambia **UNA** cosa, si rilancia lo script, si annota l'esito. **Poi** si passa alla manopola successiva — mai due cambi insieme, altrimenti non si sa quale ha fatto la differenza.

I criteri di giudizio (a occhio, non automatici):

La risposta è **corretta**?

La **citazione** punta al chunk giusto?

Quanto costa in **token**?

> Il giudice è **lo studente**, non una metrica automatica — le metriche automatiche (RAGAS) arrivano a M3.



La query set e la scheda

Le stesse 4 domande, per ogni giro:

- il codice **E07** sulla centralina
- i giorni di reso durante i **saldi**
- il pagamento in **contrassegno**
- il **contributo** per la postazione

scheda_esperimenti.md

È il **deliverable** della giornata, non il codice: una riga per giro, prima di cambiare la manopola successiva.

Budget: un giro di 4 query costa **1-2 millesimi di dollaro** — anche con molti giri, l'intera giornata resta sotto pochi centesimi.



⌨️ Lab 1 — A/B di prompt

⌨️ **TOCCA A VOI — docente in affiancamento**

`working/rag.py` · 90 minuti

poi ☕ pausa 15'
poi il reveal

`GROUNDING`
"morbido" / "severo" (TODO 1: scrivete la variante severa)

`CITAZIONE`
"breve" / "estesa" (TODO 2: scrivete il formato esteso)

`CON_COT`
True / False (TODO 3: scrivete l'istruzione di CoT)

Consegna

Provate almeno morbido/severo e no-CoT/CoT sulle 4 query. **Ogni giro** va annotato in `scheda_esperimenti.md` prima di cambiare manopola. Il vostro `rag.py` di L23? Non sovrascrivete il file: riportate solo le vostre personalizzazioni (dettagli in `CONSEGNA.md`).

Fatto quando

La scheda dice quale prompt vince e **su quali query si stacca** dalle altre.



📊 Al rientro — una query che fallisce con ogni prompt

Qualcuno l'avrà notato:

«Quanto vale il contributo per la postazione?»

Con **qualsiasi** variante di prompt provata — morbido, severo, CoT, citazione estesa — la risposta resta sempre: *«Non lo trovo nei documenti»*.

Avete cambiato il COME rispondere

Adesso cambiate **COSA** arriva prima di rispondere — il **retrieval**, non il prompt.



⌨ Lab 2 — tuning del retrieval

⌨ **TOCCA A VOI — docente in affiancamento**

`working/rag.py (stesso file) · 80 minuti`

si torna alle slide
per il caso studio

`K · SOGLIA`

Costanti nel main: si girano **senza scrivere codice**, si rilancia lo script.

`TODO 4 · reindicizza_chunks(strategia, size)`

Combina `aikit.chunk` + `aikit.vectorstore` — non è codice nuovo, è L22 richiamato da qui. Si invoca girando la manopola `REINDICIZZA` nel main.

Fatto quando

Almeno una modifica cambia **visibilmente** la risposta, e sapete spiegare perché guardando `cerca_chunk` — non solo la risposta finale.



Ultimo blocco — lo sceglie il docente

30 minuti, tre strade possibili in base a com'è andata la mattinata:

Confronto delle **schede** tra studenti

Q&A aperto, anche sulle lezioni passate

Esercizio **extra** per chi è avanti

> Qualunque strada scelga il docente, si chiude sempre col caso studio della giornata.



Il caso studio, risolto

Il documento sulla postazione spezza l'informazione in due chunk consecutivi:

`guida-smart-working-003 · score 0.396`

L'**importo** (250€) — 8° per score: fuori dai top-4

`guida-smart-working-004 · score 0.639`

La **scadenza** (60 giorni) — entra nei top-4

Due fix diversi, stesso esito:

k: 4 → 8

Risolto: entrambi i chunk arrivano, **due fonti** citate. +70% token input.

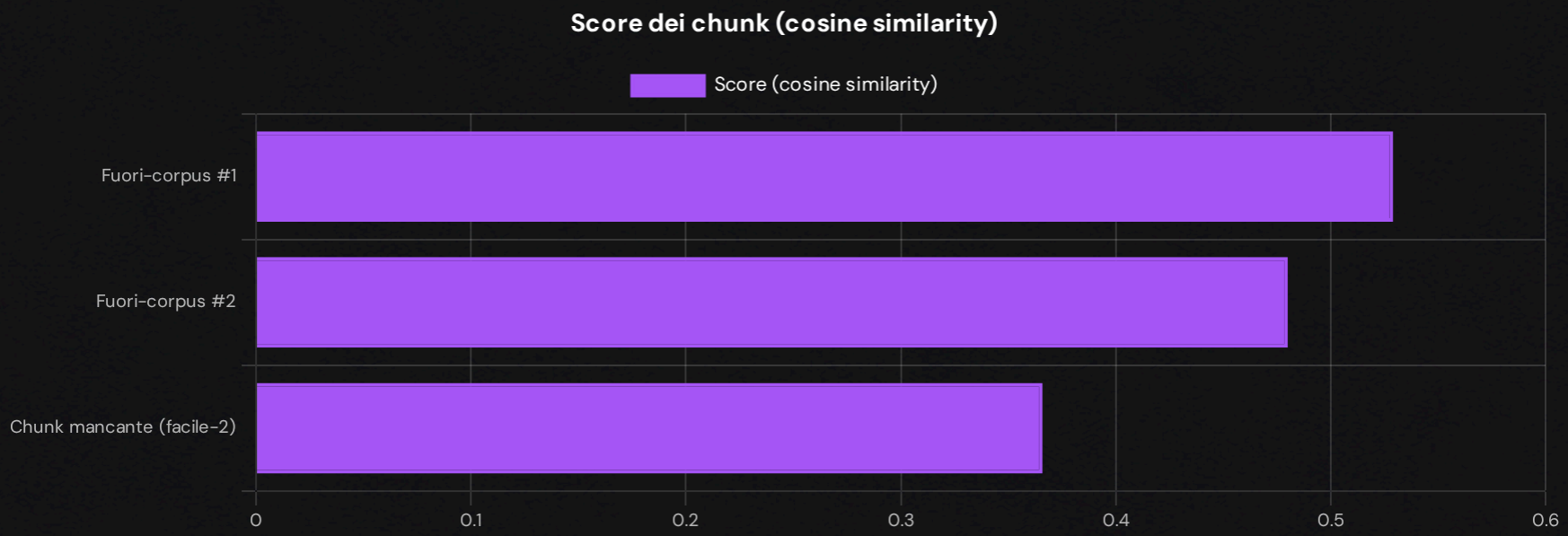
chunk size: 500 → 800

Risolto: i due dettagli in **un chunk solo**, meno token, una fonte.

> Stesso esito, due compromessi diversi: non esiste "la" soluzione giusta.



Perché la soglia non basta, qui



Nessuna soglia unica separa i due casi

Il chunk **rilevante** mancante (0.396) scora **meno** dei chunk **irrilevanti** della domanda fuori-corpus (0.50-0.57). Qui il fix è `k` o il chunk size, non lo score minimo — gli score coseno non sono calibrati (L20).

Cosa portate a casa

`rag.py` esteso e la **scheda** compilata

Prompt e retrieval sono leve diverse: un problema di retrieval non si ripara nel prompt

Il **metodo** (una manopola alla volta, tutto annotato) è quello che il Progetto Modulo 2 darà per acquisito

Martedì (L25): Structured Output

Si chiude la fase di **RAG artigianale** del modulo: si passa a output validato con schemi.



Grazie

L25 · Structured Output · martedì 21 luglio

